

2026 年 5 月 12 日 Version 2.0

ハッカソン Processing 音源サブ システム 基本設計・詳細設計 改良版

チーム 23
25G1021 : 梅澤 颯太

目次

第 1 章	本書の位置づけ	1
1.1	目的	1
1.2	担当範囲	1
1.3	評価基準への対応方針	2
第 2 章	システムの基本設計	3
2.1	機能一覧	3
2.2	システム構成図	4
2.3	パート対応	4
第 3 章	操作インターフェースと状態遷移	6
3.1	操作インターフェース	6
3.2	状態遷移	7
第 4 章	詳細設計	9
4.1	ハードウェア設計	9
4.2	ソフトウェア設計	10
4.2.1	ファイル構成	10
4.2.2	NOTE パケット設計	11
4.2.3	オブジェクト設計	12
4.3	音色設計	13
4.3.1	金管音色	13
4.3.2	リズム音色	14
第 5 章	処理フローの設計	16
5.1	起動フロー	16
5.2	NOTE 受信フロー	16
5.3	パケット検査フロー	16
5.4	ボイス管理フロー	17
5.5	例外処理	17
第 6 章	テスト設計とループリック対応	19

6.1	テスト設計	19
6.2	ループリック対応	20
第 7 章	生成 AI の利用について	22

第 1 章 本書の位置づけ

1.1 目的

本書は、チーム 23「Arduino オーケストラ」における梅澤担当範囲、すなわち Processing 側の NOTE 受信、音色合成、発音管理、最小 UI、ログ出力を対象とした基本設計・詳細設計である。

初版の設計書では、金管音色の方針は示せていたが、ループリックで要求される機能一覧、操作インターフェース、状態遷移、ハードウェア接続、NOTE 受信処理、ボイス管理、例外処理の粒度に不足があった。本改良版では、既に作成した `work/umezawa/hck/week3/orchestra_processing_final/` の実装内容を正として、実装と矛盾しない設計書へ更新する。

1.2 担当範囲

Processing は、指揮棒の加速度を直接処理しない。加速度から拍・テンポ・強弱を推定する処理は Arduino 側 `node_01` と `node_02-05` の担当である。Processing 側は、楽器ノードが USB Serial で送る NOTE パケットを受け取り、`noteNumber`, `velocity`, `durationMs` に従って音を鳴らす。

表 1.1: 梅澤担当範囲と本書での対応

WBS	タスク	本書での設計箇所
117	音色合成 基本設計	第 2 章の機能一覧、第 3 章のパート対応、第 4 章の音色設計
124	NOTE を受け取って鳴らす仕組み	第 4 章の NOTE パケット設計、第 5 章の受信フロー

WBS	タスク	本書での設計箇所
125	4 種類の楽器パートを鳴らす管理	第 3 章の <code>partId</code> 対応, 第 4 章の <code>PartManager</code> 設計
126	金管音色の具体実装	第 4 章の <code>BrassVoice</code> , 倍音比率, <code>ADSR</code> 初期値
254	UI / モード表示	第 3 章の操作インターフェース, 第 4 章の <code>UIView</code> 設計

1.3 評価基準への対応方針

本書は, システム設計書ループリックの 5 項目に対応するため, 以下を必ず記載する.

- ・ 基本設計: 機能一覧とシステム構成図
- ・ 基本設計: 操作インターフェースと状態遷移
- ・ 詳細設計: ハードウェア設計
- ・ 詳細設計: ソフトウェア設計
- ・ 詳細設計: 処理フローの設計

第 2 章 システムの基本設計

2.1 機能一覧

Processing 音源サブシステムを F6 として定義し、塩澤担当の F4「発音情報配信」の受け口以降を担当する。表 2.1 に機能一覧を示す。

表 2.1: Processing 音源サブシステムの機能一覧

ID	機能	役割	WBS
F6.1	Serial 接続管理	Processing 起動時に Serial ポート一覧を表示し、画面上のポートをクリックして接続する。切断時は再接続待ちへ戻す。	124
F6.2	NOTE フレーム同期・復号	USB Serial のバイト列から magic=0x4F52 を探し、20 byte 固定長 NOTE に復号する。	124
F6.3	パート設定管理	期待 partId を 0x02--0x05 から選び、誤接続時は発音せず警告とログを残す。検証用に ALL モードも持つ。	125
F6.4	ボイス管理	NoteOn, NoteOff, durationMs 経過による自動消音、同時発音数上限、古い Voice の解放を管理する。	125
F6.5	音色生成	0x02--0x04 は金管音色、0x05 はリズム音色として合成し、Minim の AudioOutput へ出力する。	117/126

ID	機能	役割	WBS
F6.6	UI 表示・操作	接続状態, 期待 partId, 受信数, 破棄数, 発音中 Voice 数, 波形, 警告を表示し, テスト発音やミュートを操作できるようにする.	254
F6.7	ログ・検証支援	受信時刻, NOTE 内容, 破棄理由, 発音開始を CSV に記録し, 結合試験で欠落や誤接続を確認できるようにする.	124/125

2.2 システム構成図

本番構成は, 楽器ノード 1 台に対して Mac 1 台を USB Serial で直結する構成である. Processing は自分に接続された楽器ノードから NOTE を受け, 音声出力を行う.

Listing 2.1 Processing 音源サブシステムを含む全体構成

```

User conducts
  |
  v
node_01: conductor, CTRL/BEAT sender
  |
  | UDP multicast CTRL/BEAT
  v
node_02 -- USB Serial NOTE --> Mac 1 Processing final --> Brass 1
node_03 -- USB Serial NOTE --> Mac 2 Processing final --> Brass 2
node_04 -- USB Serial NOTE --> Mac 3 Processing final --> Brass 3
node_05 -- USB Serial NOTE --> Mac 4 Processing final --> Rhythm

```

2.3 パート対応

Processing は表 2.2 の対応で partId を解釈する.

表 2.2: partId と音色・役割の対応

ノード	partId	楽譜上の役割	Processing 側音色
node_02	0x02	主旋律	Brass 1. 基準となる金管音色.
node_03	0x03	輪唱パート 1	Brass 2. 金管音色を共有し, 音量差で区別する.
node_04	0x04	輪唱パート 2	Brass 3. 金管音色を共有し, 必要に応じて調整する.
node_05	0x05	ベース・リズム伴奏	RhythmVoice. 短い減衰を持つリズム音として鳴らす.

第3章 操作インターフェースと状態遷移

3.1 操作インターフェース

Processing の利用者は、観客ではなく、デモ準備者または結合試験担当者である。したがって UI は演奏表現ではなく、接続確認、異常検出、安全停止を目的とする。

表 3.1: Processing 側操作インターフェース

操作・表示	対象	システム挙動
起動	Mac 担当者	Serial ポート一覧を取得し、画面左に接続候補として表示する。
ポートクリック	Mac 担当者	選択した Serial ポートを開き、成功したら Ready へ遷移する。
1-4 キー	Mac 担当者	期待 partId を 0x02-0x05 に切り替える。本番では Mac ごとに固定する。
a キー	検証担当者	ALL モードへ切り替え、0x02-0x05 の全パートを受け付ける。
r キー	Mac 担当者	Serial を閉じ、ポート一覧を再列挙する。
d キー	Mac 担当者	Serial を切断し、PortSelect へ戻す。
m キー	Mac 担当者	ミュートを切り替える。ミュート時は既存 Voice を Release する。

操作・表示	対象	システム挙動
t キー	Mac 担当者	選択中 partId のテスト音を鳴らし、スピーカと音量を確認する.
g キー	検証担当者	疑似 NOTE フレームを内部注入し、Arduino なしで受信処理を確認する.
画面表示	全員	接続状態, 期待 partId, 受信数, 破棄数, 欠落数, Voice 数, 波形, 警告を表示する.

3.2 状態遷移

Listing 3.1 Processing 音源サブシステムの状態遷移

```

Boot
-> PortSelect : setup completed and serial ports listed
PortSelect
-> Ready      : user clicks a serial port and open succeeds
-> Error      : serial open fails
Ready
-> Playing    : first valid NOTE received
-> PortSelect : user presses r or d
Playing
-> Ready      : no NOTE for 1000 ms, serial still open
-> PortSelect : user presses r or d
-> Error      : serial exception
Muted
-> Playing    : user presses m again
-> PortSelect : user presses r or d
Error
-> PortSelect : user presses r

```

表 3.2: 状態定義

状態	意味	主な処理
Boot	起動直後	画面, Minim, AudioOutput, Logger, PartManager, Serial-FrameReader を初期化する.
PortSelect	Serial ポート選択待ち	ポート一覧を表示し, クリック入力を待つ.
Ready	接続済み・NOTE 待ち	Serial バッファを空にし, NOTE の magic 同期を待つ.
Playing	正常受信・発音中	NOTE 復号, partId 検査, Voice 生成, ログ出力, 画面更新を行う.
Muted	受信継続・発音停止	新規 Voice は作らず, 既存 Voice を Release する.
Error	接続異常	Serial を閉じ, 未処理キューと受信バッファをクリアし, 警告を表示する.

第 4 章 詳細設計

4.1 ハードウェア設計

Processing 側では回路を追加しないが、ループリックのハードウェア設計項目に対応するため、接続される構成要素と故障時動作を明確化する。

表 4.1: Processing 側関連部品

ID	要素	数量	役割
P1	Arduino UNO R4 WiFi	4	node_02-05. 楽譜進行後, NOTE パケットを USB Serial で送信する.
P2	Mac または PC	4	Processing 実行機. 本番は 1 台につき 1 つの Serial ポートを開く.
P3	USB Type-C ケーブル	4	Arduino と Mac の給電および Serial 通信を兼ねる.
P4	スピーカまたは内蔵スピーカ	4	Processing の AudioOutput を再生する.
P5	Processing Minim	4 + 4 環境	NOTE を受けて音を合成する実行環境.

表 4.2: Processing 側ハードウェア接続設計

項目	設計
接続方式	node_02-05 と Mac 1-4 を USB Type-C で 1 対 1 接続する.
通信方式	USB CDC Serial. Arduino 側は <code>Serial.write()</code> で NOTE を 20 byte 固定長バイナリとして送る.
ボーレート	115200 bps. Arduino 側の <code>NoteSender</code> 設定と統一する.
フレーミング	全 NOTE は先頭 2 byte の <code>magic=0x4F52</code> を同期マークとする. Serial 上では <code>0x52 0x4F</code> の順に現れる.
音声出力	各 Mac の既定オーディオ出力へ送る. 本番前に OS 音量と Processing 側ゲインを確認する.
誤接続対策	Mac ごとに期待 <code>partId</code> を設定し, 異なる <code>partId</code> は発音せず警告とログを残す.
切断時動作	Serial 切断または例外時はポートを閉じ, 受信キューとバッファをクリアし, 全 Voice を停止する.

4.2 ソフトウェア設計

4.2.1 ファイル構成

実装は `work/umezawa/hck/week3/orchestra_processing_final/` に配置する.

Listing 4.1 Processing 実装ファイル構成

```

orchestra_processing_final/
|-- orchestra_processing_final.pde  // setup(), draw(), serialEvent(),
    key/mouse input
|-- Config.pde                      // serial, protocol, partId, voice
    constants

```

```

|-- Protocol.pde // NotePacket, little-endian decoder,
    test frame
|-- SerialFrameReader.pde // byte buffer and magic sync
|-- PartManager.pde // part voice allocation and voice
    limits
|-- Voice.pde // Voice, BrassVoice, RhythmVoice
|-- UIView.pde // status screen, clickable port list
    , waveform
|-- Logger.pde // CSV logging
|-- README.md // operation note
`-- sketch.properties // Processing Java mode

```

4.2.2 NOTE パケット設計

Processing は表 4.3 の 20 byte を固定長 NOTE として読む。Processing 側で独自形式を作らず、塩澤設計の OrcProtocol に合わせる。

表 4.3: NOTE パケットの受信側解釈

offset	field	type	Processing 側の扱い
0	magic	uint16	0x4F52 以外なら同期探索を続ける。
2	version	uint8	0x01 以外なら破棄し、version error としてログする。
3	type	uint8	3 のときだけ NOTE として処理する。CTRL/BEAT は発音しない。
4	seq	uint32	partId ごとに前回値を持ち、重複・欠落を検出する。
8	timestampMs	uint32	Arduino 側の送信時刻。ログへ保存し、遅延評価の参考にする。
12	partId	uint8	0x02–0x05。期待 partId または ALL と一致する場合だけ発音する。

offset	field	type	Processing 側の扱い
13	noteNumber	uint8	MIDI ノート番号として周波数へ変換する。0 は停止相当として扱う。
14	velocity	uint8	0-127 を 振 幅 へ 変 換 す る。 MASTER_GAIN を 掛 け て 過 大 音 量 を 避ける。
15	gate	uint8	1=NoteOn, 0=NoteOff. その他は破棄する。
16	durationMs	uint16	NoteOn 時の自動停止予定時刻を決める。0 の場合は最小長を使う。
18	reserved	uint8[2]	現時点では未使用。

4.2.3 オブジェクト設計

表 4.4: Processing 側オブジェクト設計

クラス・構造体	主なフィールド・メソッド	役割
NotePacket	NOTE の各フィールド	20 byte フレームから復号した発音指令を表す。
SerialFrameReader	pushByte(), pollPacket(), clear()	Serial 入力を蓄積し, 0x52 0x4F を探して 20 byte を切り出す。
PacketQueue	offer(), poll()	実装では ConcurrentLinkedQueue<NotePacket> を用いる。Serial スレッドと draw スレッドを分離する受信キュー。

クラス・構造体	主なフィールド・メソッド	役割
PartManager	handleNote(), noteOn(), noteOff(), update()	Voice の生成, 同時発音上限, duration 経過による自動停止を管理する.
Voice	start(), update(), release(), stop()	1 つの発音単位を表す基底クラス. partId, noteNumber, amp, 開始時刻, 停止予定時刻を持つ.
BrassVoice	attackMs=20, decayMs=40, sustain=0.75, releaseMs=60	金管 1-3 の音色を生成する. 倍音テーブルから作った波形を使う.
RhythmVoice	rhythmDecay(), rhythmFreq(), rhythmWave()	node_05 のリズム音色を生成する. noteNumber により Kick/Snare/Hi-hat 風に分ける.
UIView	draw(), drawPortButton(), portIndexAt()	状態表示, クリック式ポート選択, 波形モニタ, 操作説明を描画する.
Logger	logPacket(), logEvent(), close()	結合試験用 CSV を保存する.

4.3 音色設計

4.3.1 金管音色

金管音色は, 基音に複数の倍音を重ねた波形を `WavetableGenerator.gen10()` で作る. 初期値は表 4.5 とする.

ADSR は表 4.6 の初期値とする. 音符全体を単純な直線減衰にすると金管らしさ

表 4.5 金管音色の初期倍音比率

倍音	1	2	3	4	5	6	7
振幅比	1.00	0.70	0.50	0.30	0.20	0.10	0.05

が落ちるため、Attack, Decay, Sustain, Release を分ける。

表 4.6: 金管 ADSR 初期値

項目	初期値	意味	調整方針
Attack	20 ms	息の入りの立ち上がり	遅い場合は 10 ms へ短縮する。
Decay	40 ms	最大音量から持続音量へ移る時間	音が硬い場合は延長する。
Sustain	0.75	音符中の持続音量比	velocity に応じてスケールする。
Release	60 ms	息を止めた後の余韻	音の切れが悪い場合は 40 ms へ短縮する。

4.3.2 リズム音色

node_05 はリズム伴奏であるため、金管音色ではなく短い減衰音として扱う。現在の実装では、noteNumber に応じて周波数、波形、減衰時間を切り替える。

表 4.7: リズム音色の設計

noteNumber	音色カテゴリ	減衰時間	合成方針
36–37	Kick/Bass	150 ms	低い SINE 波でベース感を作る。
38–40	Snare	95 ms	中域の SQUARE 波で短い打音にする。

noteNumber	音色カテゴリ	減衰時間	合成方針
41-44	Hi-hat	45 ms	高域の SAW 波で鋭いリズム音にする.
その他	Click	70 ms	未定義音でも無音にならないよう短いクリック音にする.

第 5 章 処理フローの設計

5.1 起動フロー

1. `setup()` で画面, `Minim`, `AudioOutput`, 設定値, `Logger`, `SerialFrameReader`, `PartManager`, `UIView` を初期化する.
2. `lastSeqByPart` を `-1` で初期化し, `part` 別 `seq` 管理を開始できる状態にする.
3. `Serial.list()` でポート一覧を取得し, `PortSelect` 状態にする.
4. ユーザがポート一覧をクリックしたら `connectSerial(index)` を呼ぶ.
5. 接続成功なら `Ready`, 失敗なら `Error` とし, 画面とログに理由を残す.

5.2 NOTE 受信フロー

Listing 5.1 NOTE 受信から発音までの処理フロー

```
1 serialEvent(serial):
2   while serial has bytes:
3     frameReader.pushByte(serial.read())
4     while frameReader.pollPacket() returns packet:
5       packetQueue.offer(packet)
6
7 draw():
8   drainPackets()
9   partManager.update()
10  ui.draw()
11
12 drainPackets():
13   while packetQueue has packet:
14     handlePacket(packet)
```

5.3 パケット検査フロー

Listing 5.2 handlePacket の検査フロー

```
1 handlePacket(packet):
2     if version != 1: drop("version error")
3     if type != NOTE: drop("non NOTE type")
4     if gate is not 0 and not 1: drop("gate error")
5     if partId is not 0x02..0x05: drop("unknown part")
6     if expectedPartId != ALL and partId != expectedPartId:
7         drop("wrong part")
8     if seq equals lastSeqByPart[partId]:
9         drop("duplicate seq")
10    if seq skips lastSeqByPart[partId] + 1:
11        missingPackets += skipped count
12    lastSeqByPart[partId] = seq
13    logger.logPacket("note", packet)
14    partManager.handleNote(packet)
```

5.4 ボイス管理フロー

1. gate=1 かつ noteNumber>0 かつ velocity>0 の場合, NoteOn として扱う.
2. gate=0, noteNumber=0, velocity=0 の場合, NoteOff 相当として扱う.
3. partId=0x02--0x04 は BrassVoice, partId=0x05 は RhythmVoice を生成する.
4. velocity を 0.0-1.0 に正規化し, MASTER_GAIN=0.55 を掛けて振幅を決める.
5. 同一 partId の非 release Voice が MAX_VOICES_PER_PART=4 以上なら, 最も古い Voice を release する.
6. durationMs が小さすぎる場合は MIN_DURATION_MS=120 を使う.
7. update() で停止予定時刻を過ぎた Voice を release し, release 完了後に unpatch して配列から削除する.

5.5 例外処理

表 5.1: Processing 側例外処理

条件	処理	目的
magic 不一致	1 byte ずつ破棄して次の 0x52 0x4F を探す.	バイトずれから復帰する.
version 不一致	パケットを破棄し, CSV に理由を残す.	旧仕様・別仕様との混在を防ぐ.
type 不一致	NOTE 以外は破棄する.	CTRL/BEAT を誤って発音しない.
gate 不正	パケットを破棄する.	不正値による無限発音を防ぐ.
partId 不明	パケットを破棄する.	未定義パートを誤発音しない.
partId 不一致	発音せず画面警告とログを残す.	Mac と Arduino の誤接続を発見する.
seq 重複	発音せずログのみ残す.	二重発音を防ぐ.
seq 欠落	発音は継続し, 欠落数を加算する.	聴感破綻を避けつつ検証可能にする.
Serial 切断	ポートを閉じ, 受信キューとバッファをクリアする.	再接続時の古いデータ混入を防ぐ.

第 6 章 テスト設計とループバック対応

6.1 テスト設計

表 6.1: Processing 側テスト設計

ID	試験項目	手順	合否基準
PT-1	ビルド確認	<code>processing-java --build</code> を実行する.	エラーなく完了する.
PT-2	NOTE 復号単体	g キーで疑似 NOTE を注入する.	受信数が増え, テスト音が鳴る.
PT-3	ポート選択 UI	Serial ポート一覧をクリックする.	接続状態が Ready になり, ポート名が表示される.
PT-4	partId 誤接続	期待 partId と違う NOTE を入力する.	発音せず wrong part と表示・ログ出力する.
PT-5	NoteOn 自動停止	<code>durationMs=500</code> の NOTE を入力する.	約 500 ms 後に release され, 鳴りっぱなしにならない.

ID	試験項目	手順	合否基準
PT-6	同時発音上限	同一 partId で 5 音以上を連続入力する.	古い Voice が release され, 上限 4 を維持する.
PT-7	リズム音色	partId 0x05 で noteNumber 36, 38, 42 を入力する.	短いリズム音として鳴る.
PT-8	Serial 再接続	a または r を押して再接続する.	キューとバッファがクリアされ, 再接続できる.
PT-9	結合演奏	node_02-05 と Mac 4 台を接続し演奏する.	欠落なく鳴り, 誤接続警告が出ない.

6.2 ループリック対応

表 6.2: システム設計書ループリックとの対応

評価項目	本書での対応	目標
基本設計：機能一覧とシステム構成図	第 2 章で FBS, Mac 4 台構成図, partId 対応表を示した.	A
基本設計：操作インターフェースと状態遷移	第 3 章でクリック式ポート選択, キー操作, 状態遷移図, 状態定義を示した.	A
詳細設計：ハードウェア設計	第 4 章で Arduino, Mac, USB, スピーカ, 通信方式, 故障時動作を示した.	A

評価項目	本書での対応	目標
詳細設計：ソフトウェア設計	第4章で実装ファイル構成, NOTE パッケージ, クラス設計, 音色パラメータを示した.	A
詳細設計：処理フローの設計	第5章で起動, Serial 受信キュー, パッケージ検査, ボイス管理, 例外処理を示した.	A

第 7 章 生成 AI の利用について

本設計書の改良にあたり、生成 AI を利用した。利用範囲は、既存設計書、レビュー資料、塩澤設計、実装済み Processing プログラムの読み取り補助、章立ての整理、ループリック項目との対応確認、文章案の作成である。

ただし、採用する前提条件、NOTE パケット仕様、Mac 4 台構成、partId 対応、音色パラメータ、テスト項目については、リポジトリ内資料および実装済みプログラムを確認したうえで判断した。生成 AI の出力はそのまま採用せず、塩澤・齋藤の仕様および梅澤担当範囲と矛盾しないよう修正した。